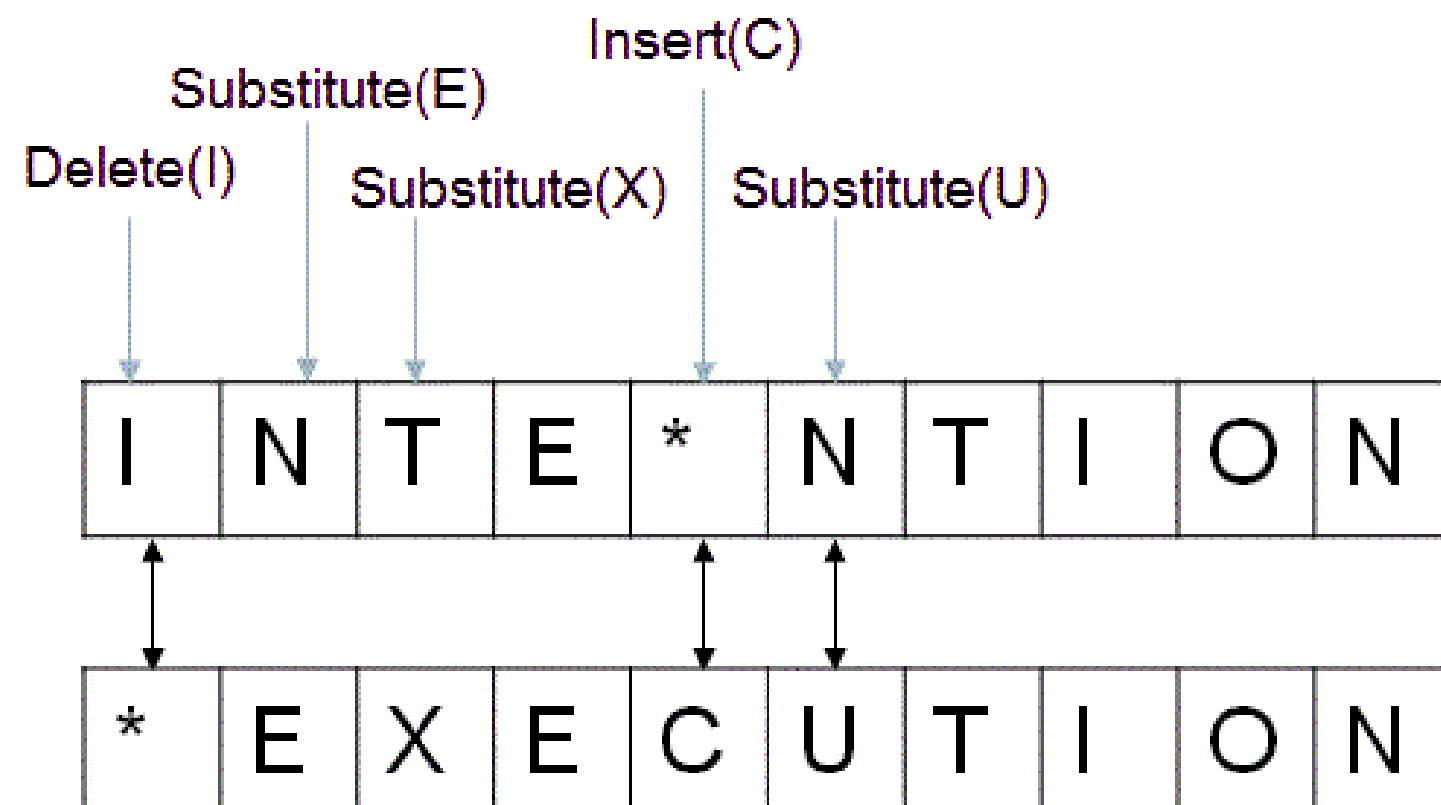


APPROXIMATE STRING MATCHING

WITH EDIT DISTANCE $\leq K$



Abdullah Irfan, 29266
Abdullah Khalid, 28084
Kashish Anil Kumar, 29105
Muskan Pawan, 28394

Problem Statement

The “WHAT”

The Problem: Approximate String Matching with Edit Distance $\leq k$

Given

a **text** $T = t_1 t_2 \cdots t_n$ of length n ,
a **pattern** $P = p_1 p_2 \cdots p_m$ of length m } over alphabet Σ
an error budget $k \geq 0$

Edit distance $d(X, Y) = \min \#$ of single-character **insertions**, **deletions**, or **substitutions** needed to turn X into Y .

e.g. $d(\text{cat}, \text{bat})=1$ $d(\text{cat}, \text{cats})=1$, $d(\text{cat}, \text{dog})=3$.

Goal — find every position where P almost occurs in T :

$$\text{Output} = \left\{ i \mid \exists j \geq i : d(P, T[i..j]) \leq k \right\}$$

A Simple Example

Pattern $P = \text{"cat"}$

Budget $k = 1$

Text $T = \text{"I saw a cat and a bat eat a rat"}$

I saw a cat and a bat eat a rat

Window	$d(P, \text{window})$	Within budget?	Edit to reach 'cat'
cat	0	yes	exact match
bat	1	yes	substitute $b \rightarrow c$
eat	1	yes	substitute $e \rightarrow c$
rat	1	yes	substitute $r \rightarrow c$
saw, and, the, ...	≥ 2	no	rejected

Output: 4 approximate occurrences of cat.

Where This Problem Shows Up

The same edit-distance problem powers a surprising range of systems.

Bioinformatics & genomics

Align noisy DNA / protein reads to a reference — mutations and sequencing errors are edits.

BLAST, BWA, Bowtie, minimap2 ($n \approx 3 \times 10^9$).

Spell-check & autocomplete

You type kat; system suggests cat, ranked by edit distance.

Google “did you mean”, browser autocorrect, hunspell.

Typo-tolerant / fuzzy search

Query “Stockholm” → still returns “Stockholm”.

Elasticsearch, Lucene, Algolia, MongoDB.

Plagiarism & code-clone detection

Find documents or source files differing by small edits / rewording.

Turnitin, MOSS (Stanford), JPlag.

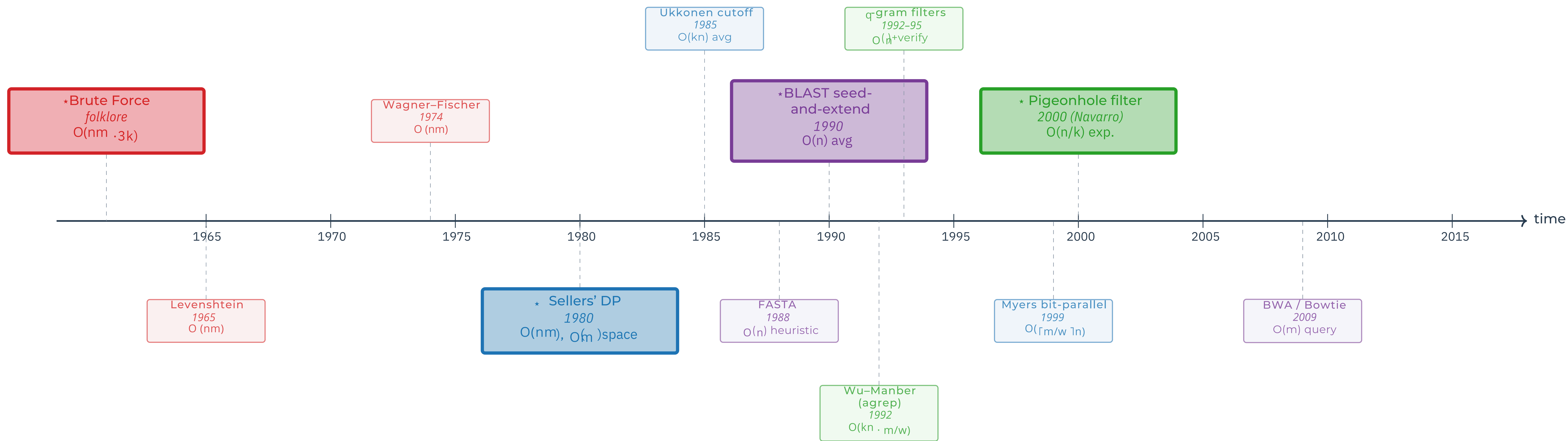
Anywhere two strings need to be compared forgivingly, this is the underlying problem.

Algorithm Selection

The “WHY”

A Short History of Approximate String Matching

twelve milestones, four paradigms, fifty years — * marks the four we benchmark



Paradigm legend: ■ Brute Force & classical DP ■ DP optimisations ■ Filter methods ■ Seed-and-Extend / heuristic | * = the four algorithms we benchmark

Why these four?

One canonical algorithm per paradigm $\Rightarrow$ the comparison spans the entire design space

Brute Force Pedagogical baseline; exponential in k — the cost of not being clever.

Sellers' DP Textbook $O(nm)$ exact DP — our ground truth for the other three

Pigeonhole filter Divide-and-conquer: split P into $k+1$ pieces, then verify candidate hits

Seed-and-Extend Heuristic behind BLAST / BWA — our only approximation; measures the speed vs. recall trade-off

One algorithm per CSE 317 paradigm: recursion, dynamic programming, divide-and-conquer, greedy / approximation.

1. Naive Brute Force

The Recursive Baseline
Every position, every error.

Recursive Branch-and-Bound

Baseline Algorithm

How It Works

Core Idea: Exhaustively explore every possible edit operation (substitution, insertion, deletion) at each mismatch, pruning any path whose accumulated error exceeds k .

1

Slide Window over T

For each starting position i in text T , align pattern P and begin recursive matching.

2

Match or Branch

If characters match, advance both pointers. On mismatch, recurse into 3 branches: substitution, insertion, and deletion.

3

Bound (Prune)

If the accumulated error count exceeds threshold k on any branch, immediately terminate that path — no further exploration.

4

Report Match

If the pattern is fully consumed with error $\leq k$, record ending position j as a valid approximate match.

Time: $O(3^m \times n)$ **Space:** $O(m + k)$ **Optimality:** **Exact (Correctness Oracle)**

Recursive Branch-and-Bound

Baseline Algorithm

Analysis & Empirical Results

3,440

Test Cases Run

synthetic + biological

2,086 μs

Mean Runtime

max: 27,693 μs

0.902

Mean Precision

avg across all cases

74

Cases Auto-Skipped

$n \cdot 3^k > 30\text{M ops}$

⚠ Scalability Limits

- Unusable past $k \approx 3$ or $n \approx 50,000$
- Worst-case: identical repeating chars ($T = \text{AAAA...}$, $P = \text{AAAA...}$) bypasses all bounding — forces full ternary recursion fan-out
- 74 high-stress cases required automatic skipping to prevent system freeze
- Exponential collapse is unavoidable at higher error budgets

✓ Key Takeaways

- Mean recall of 0.94 — highly accurate on small inputs
- Best-case $O(n)$ when pruning is aggressive (small k , diverse alphabet)
- Space-efficient at $O(m + k)$ — no explicit matrix required
- Serves as the Correctness Oracle: all advanced algorithms are cross-validated against its output

Verdict: Exact validation tool for tiny inputs — **inevitable exponential collapse** at higher k proves the necessity of DP & filtering paradigms.

2. Sellers' Algorithm

Exact Approximate Matching via DP

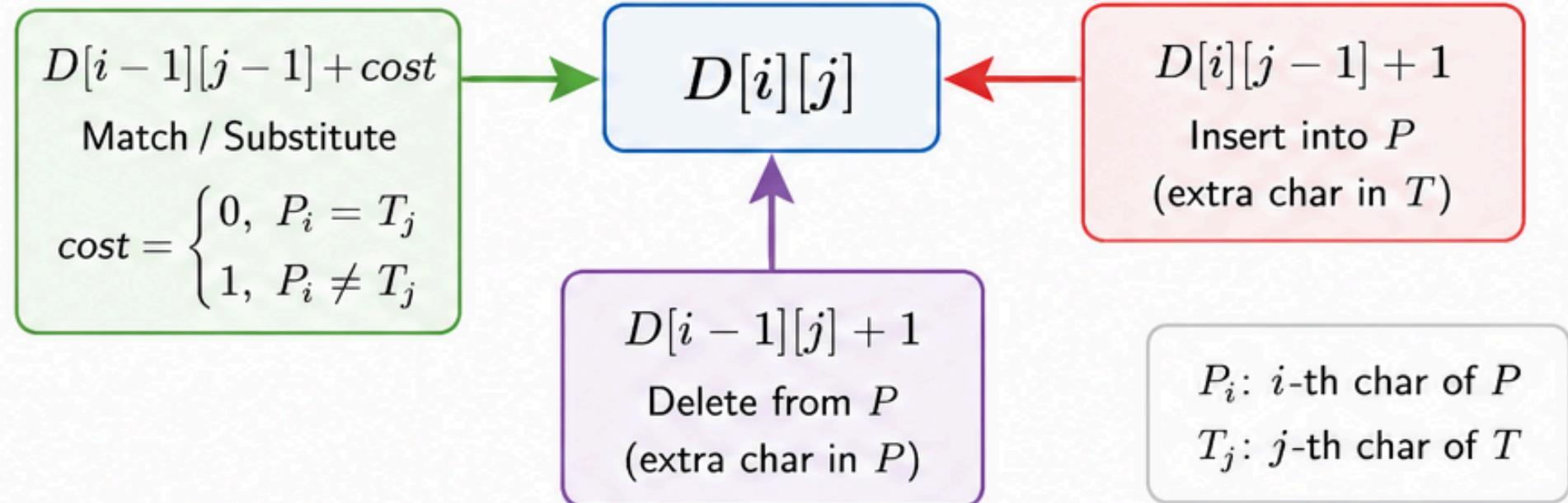
Seller's Algorithm (Approximate String Matching)

DP Idea

$D[i][j]$ = min edit distance
between first i chars of P
and any suffix of first j chars of T .

Allows pattern to start
anywhere in T .

DP Recurrence



Initialization

$D[0][j] = 0 \quad \forall j$
(pattern can start anywhere)

$D[i][0] = i \quad \forall i$
(delete all i chars of P)

Main Loop

For $j = 1 \dots n$
For $i = 1 \dots m$
Compute $D[i][j]$ using
recurrence above.

Match Rule

If $D[m][j] \leq k$
↓
Report match ending
at position j in T .
(edit distance $\leq k$)

Algorithm Flow

1 Input

Text T (length n)
Pattern P (length m)
Threshold k

2 Initialize DP

$D[0][j] = 0$
 $D[i][0] = i$

3 Fill DP Table

Compute $D[i][j]$
column by column
($j = 1 \dots n$)

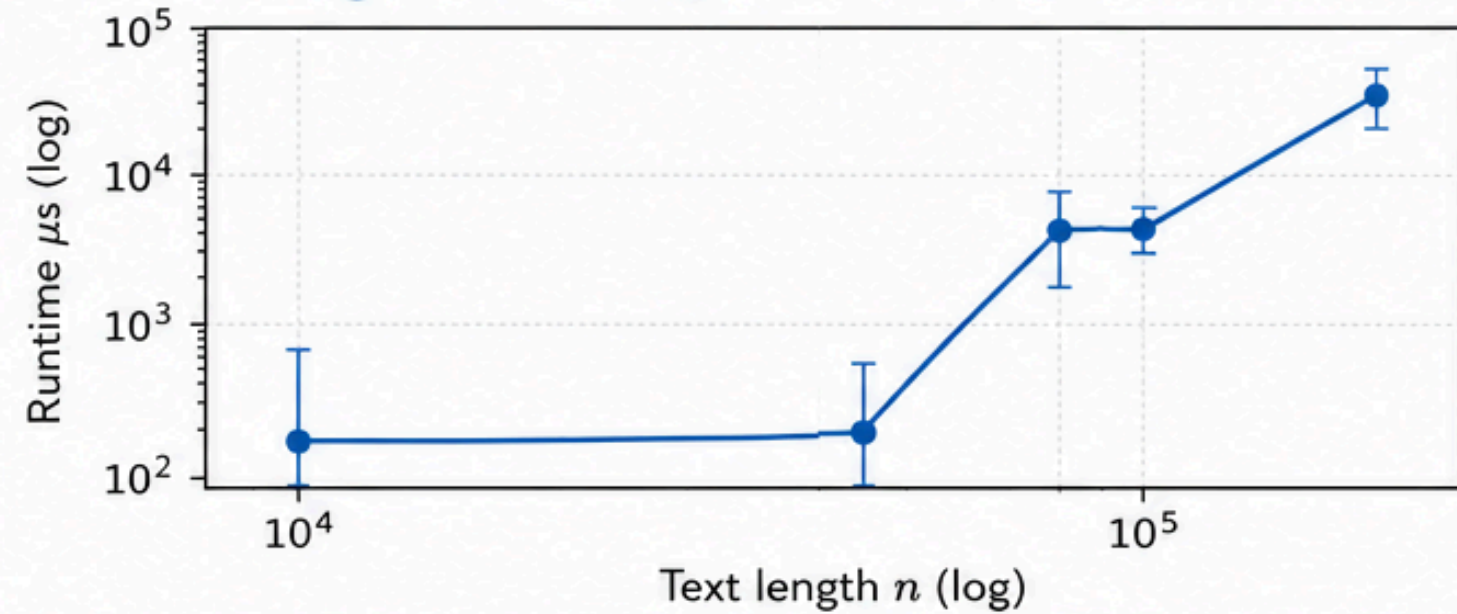
4 Report Matches

Whenever $D[m][j] \leq k$,
output end position j
and distance.

Seller's DP (Kashish) – Scaling Across All Dimensions

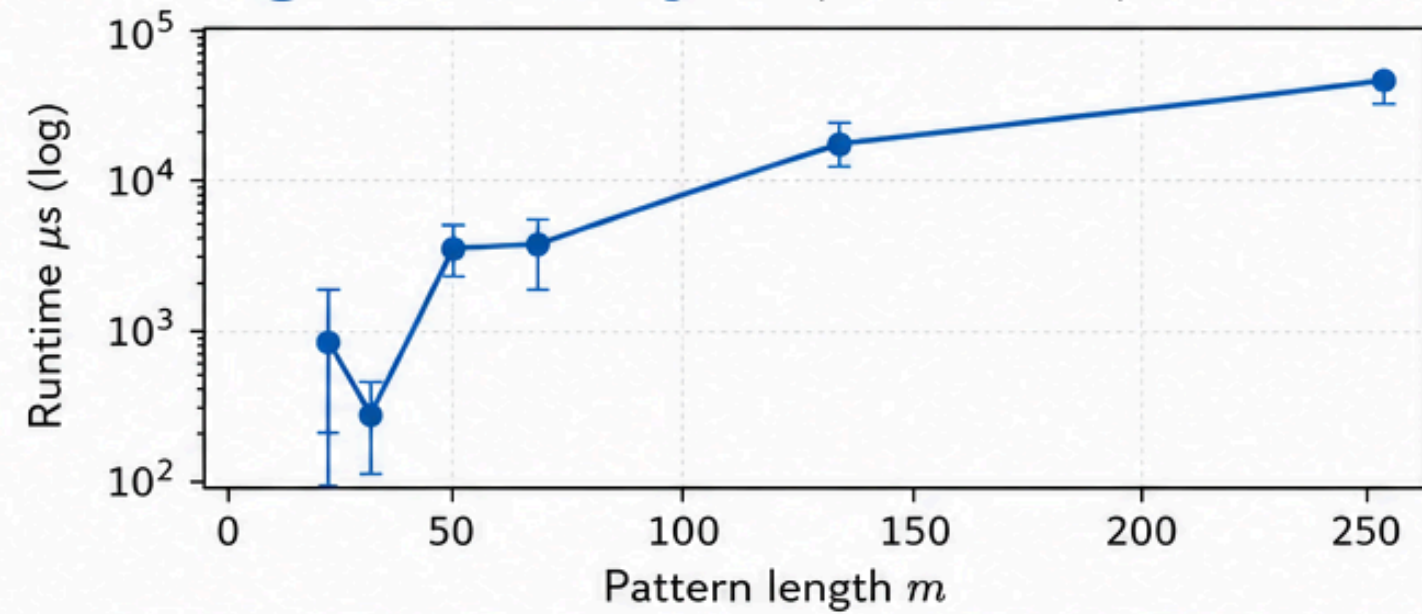
Family = DNA (n/m/k panels); mean $\pm$ SE across seeds.

1 vs Text Length n ($m = 20, k = 2$)



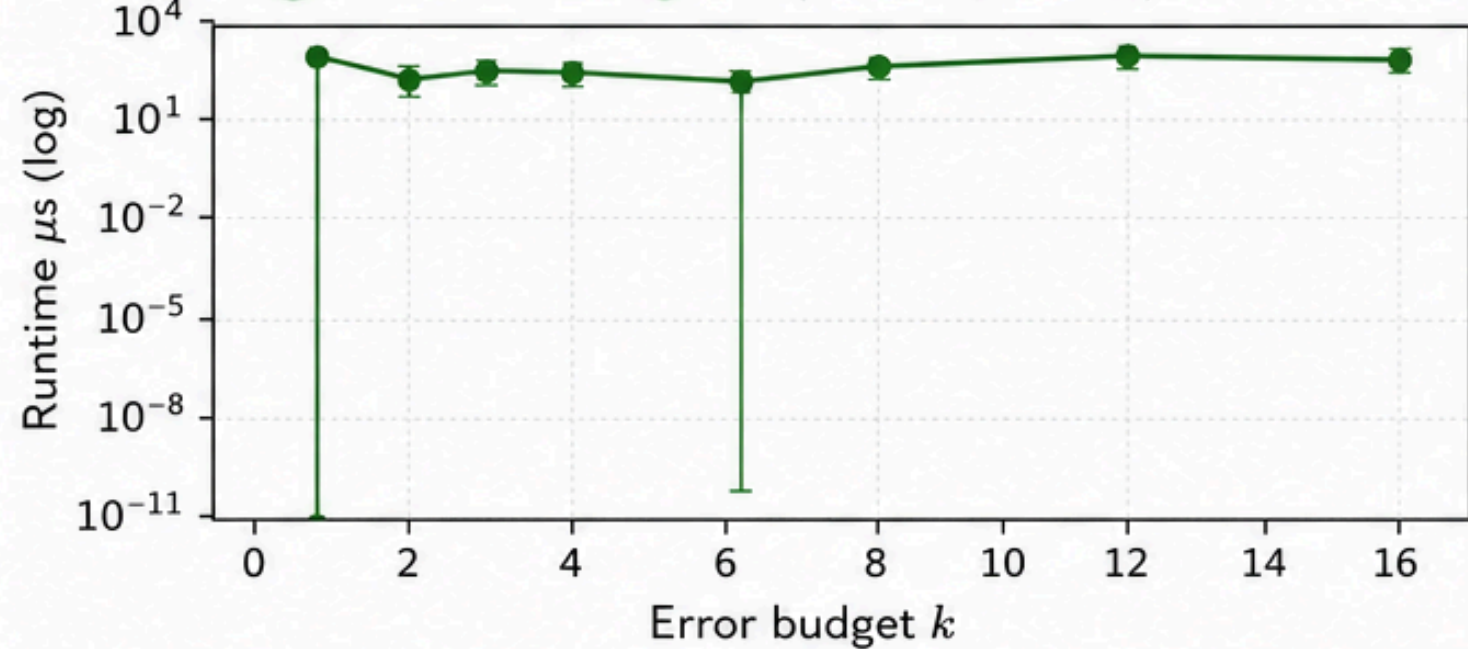
→ Runtime increases sharply with text length n .

2 vs Pattern Length m ($n = 20k, k = 2$)



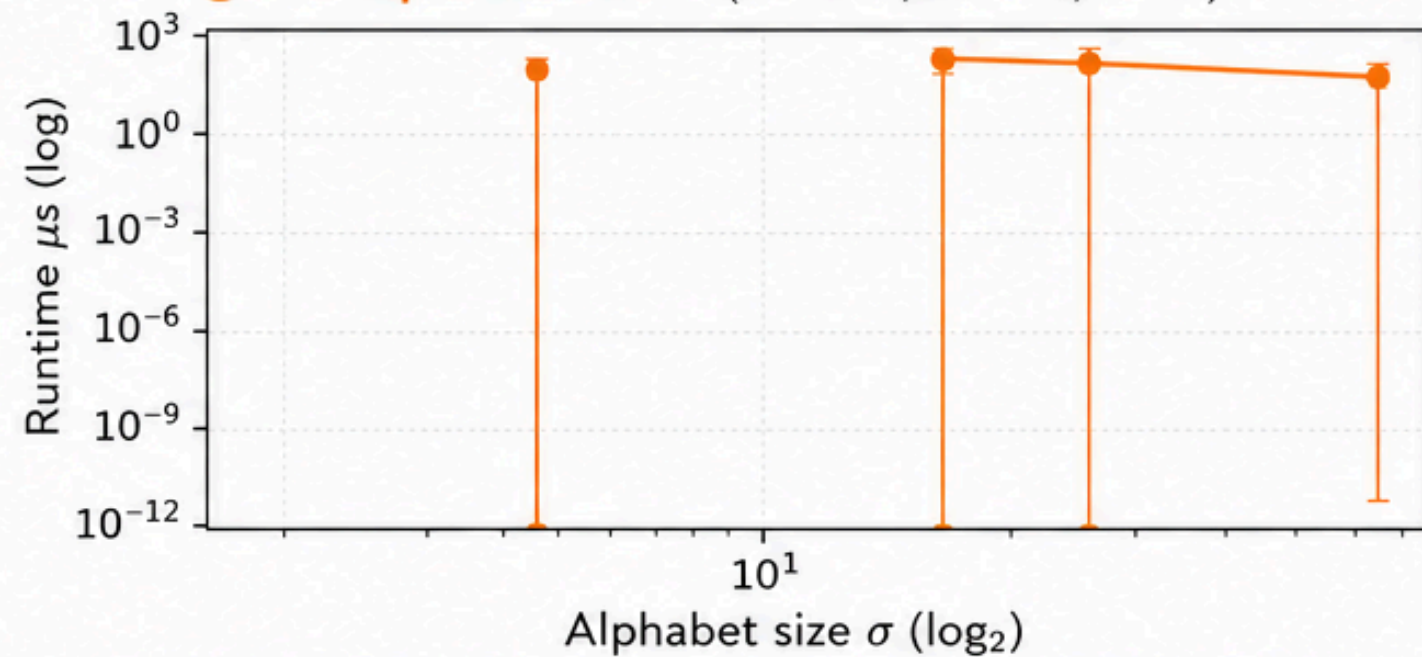
→ Runtime grows as pattern length m increases.

3 vs Error Budget k ($n = 20k, m = 32$)



→ Weak dependence on k ; runtime stays mostly stable.

4 vs Alphabet Size σ ($n = 20k, m = 20, k = 2$)



→ Runtime is nearly insensitive to alphabet size σ .

★ Seller's DP scales well: $\sim$ linear in text length and pattern length, robust to k and σ .

3. Pigeonhole Filtering

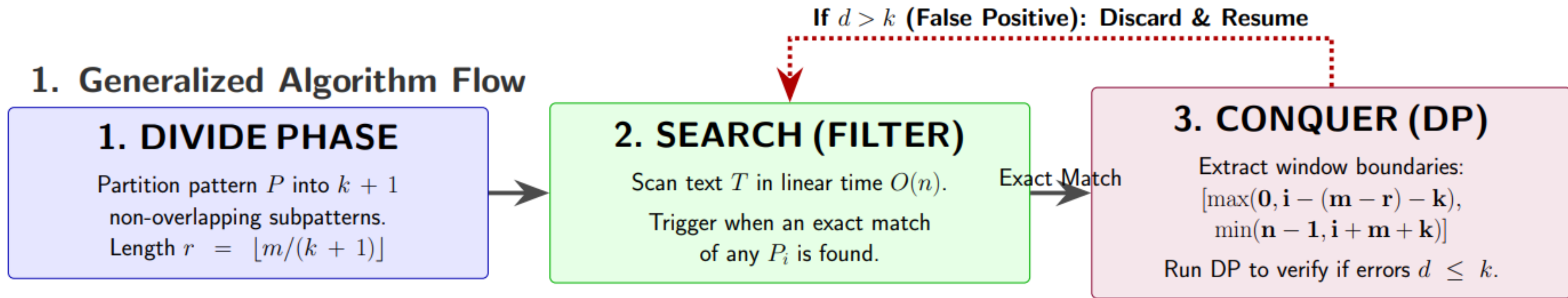
Divide, Verify, Conquer

Pigeonhole Principle Filter: How It Works

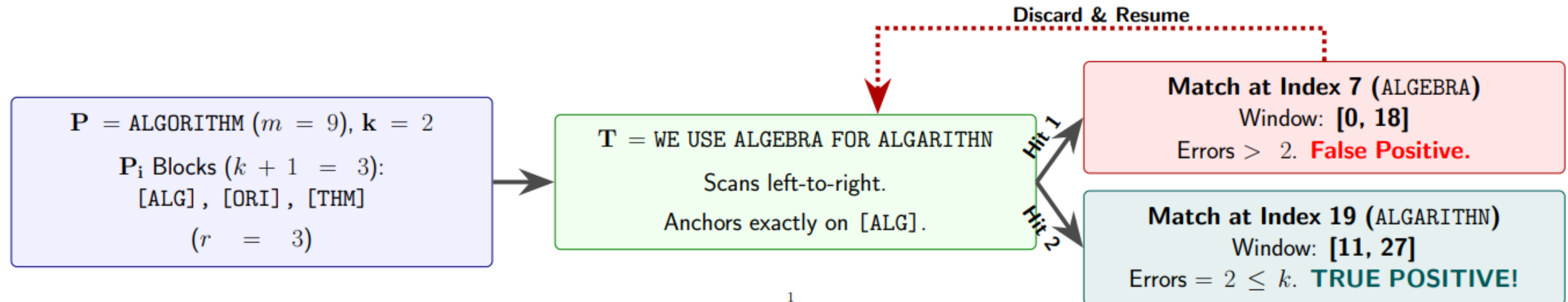
The Heuristic ("Filter, then verify"): Instead of running $O(m^2)$ DP on the entire text, mathematically guarantee an exact subpattern match to prune $> 90\%$ of the text in $O(n)$ time.

Parameters: T = Text (n), P = Pattern (m), k = Allowed Errors, r = Subpattern length.

1. Generalized Algorithm Flow



2. Specific Example Trace



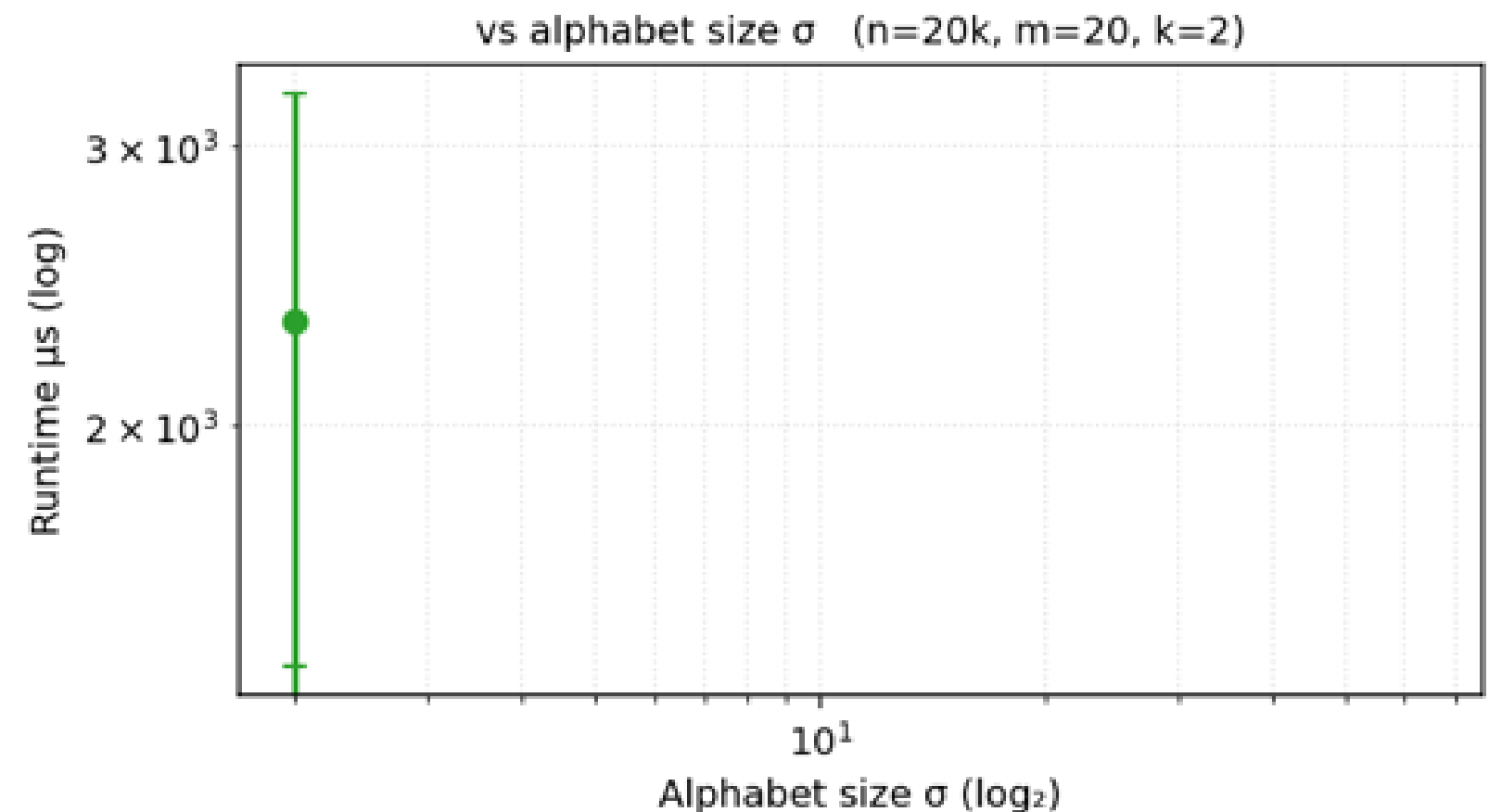
Best Case $O(n)$ = (no exact matches)

Average Case $O(n)$

Worst Case $O(n m^2)$ = ($m \sim k \Rightarrow r$ very small)

- Scanning Time: $O(n)$
- Expected Hits: $n \times \frac{1}{\sigma^r}$
- DP Time: $O(m^2)$

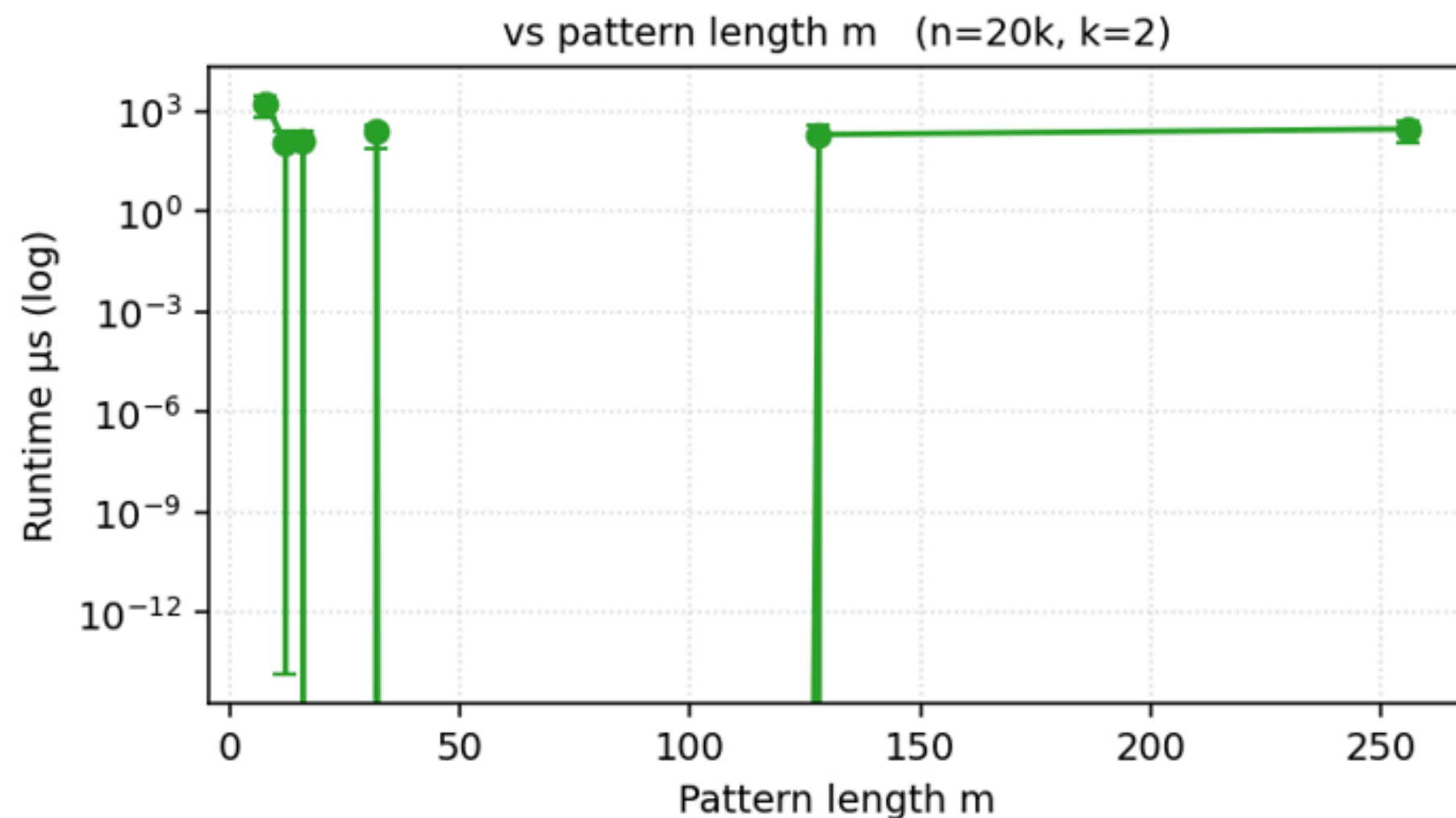
$$\text{Average Time} = O(n) + \left(n \times \frac{1}{\sigma^r} \times O(m^2) \right)$$



- If $r = 6$ in English ($\sigma = 26$), the probability is $\frac{1}{26^6}$, or about **1 in 308 million**.

Benchmark Validation: Explaining the Curves

How the C++ pigeonhole_search and verify_window functions produce the theoretical bounds.



2. vs. Pattern Length (m)

The Graph: Runtime drops rapidly, then flatlines.

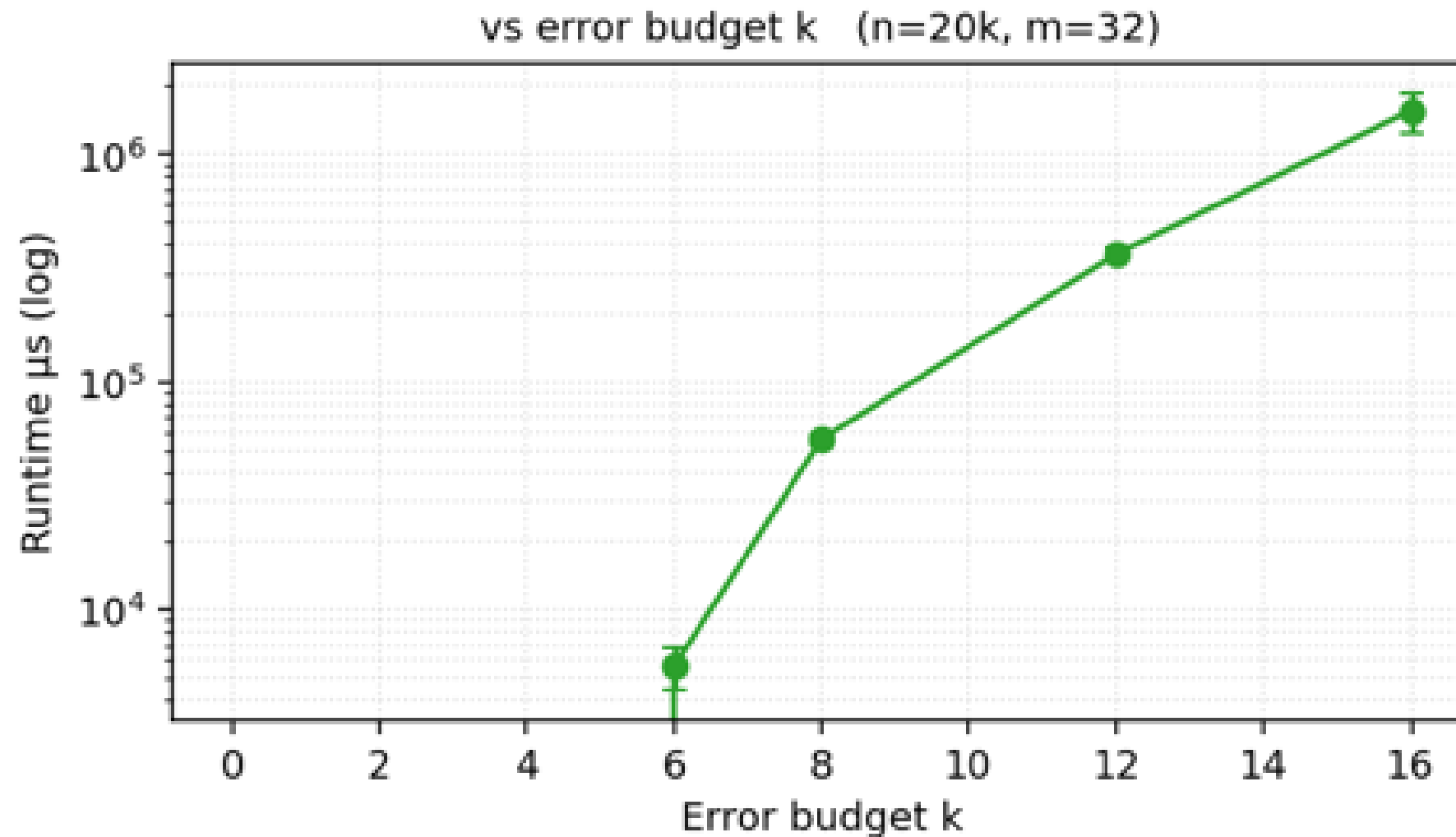
The Concept: Maximum Efficiency.

Driven by the formula $r = \max(1, m/(k+1))$.

When $m = 130$, block size $r = 43$. The probability of accidentally finding a 43-character sequence is effectively zero. False positives vanish, and efficiency is maximized.

Benchmark Validation: Explaining the Curves

How the C++ pigeonhole_search and verify_window functions produce the theoretical bounds.



$m=32$

$k=12 \Rightarrow r=2$

$k=14 \Rightarrow r=2$

$k=16 \Rightarrow r=2$

$k=18 \Rightarrow r=1$

3. vs. Error Budget (k)

The Graph: Exponential explosion in execution time.

The Concept: **The Mathematical Collapse.**

As k approaches 15, the code chops the pattern into 16 pieces. Block size shrinks to $r = 2$.

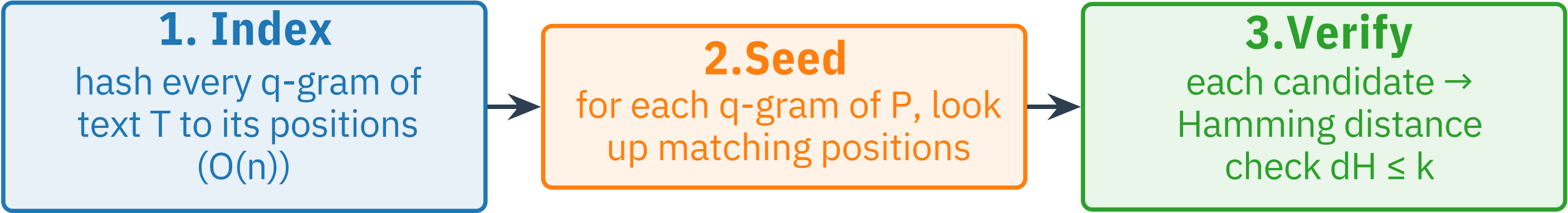
In DNA, 2-letter blocks (like "CG") match everywhere. The filter becomes pure noise, triggering the $O(m^2)$ DP matrix constantly.

4. Seed-and-Extend

A BLAST-Style Heuristic

Seed-and-Extend: How It Works

“Filter, then verify” — BLAST-style (Altschul et al., 1990).



Worked example: $T = \text{ACGTACAT}$, $P = \text{ACGT}$, $k = 1$, $q = 2$.

1. Index of T	
q -gram	positions
AC	{0, 4}
CG,GT,TA	{1}, {2}, {3}
CA,AT	{5}, {6}

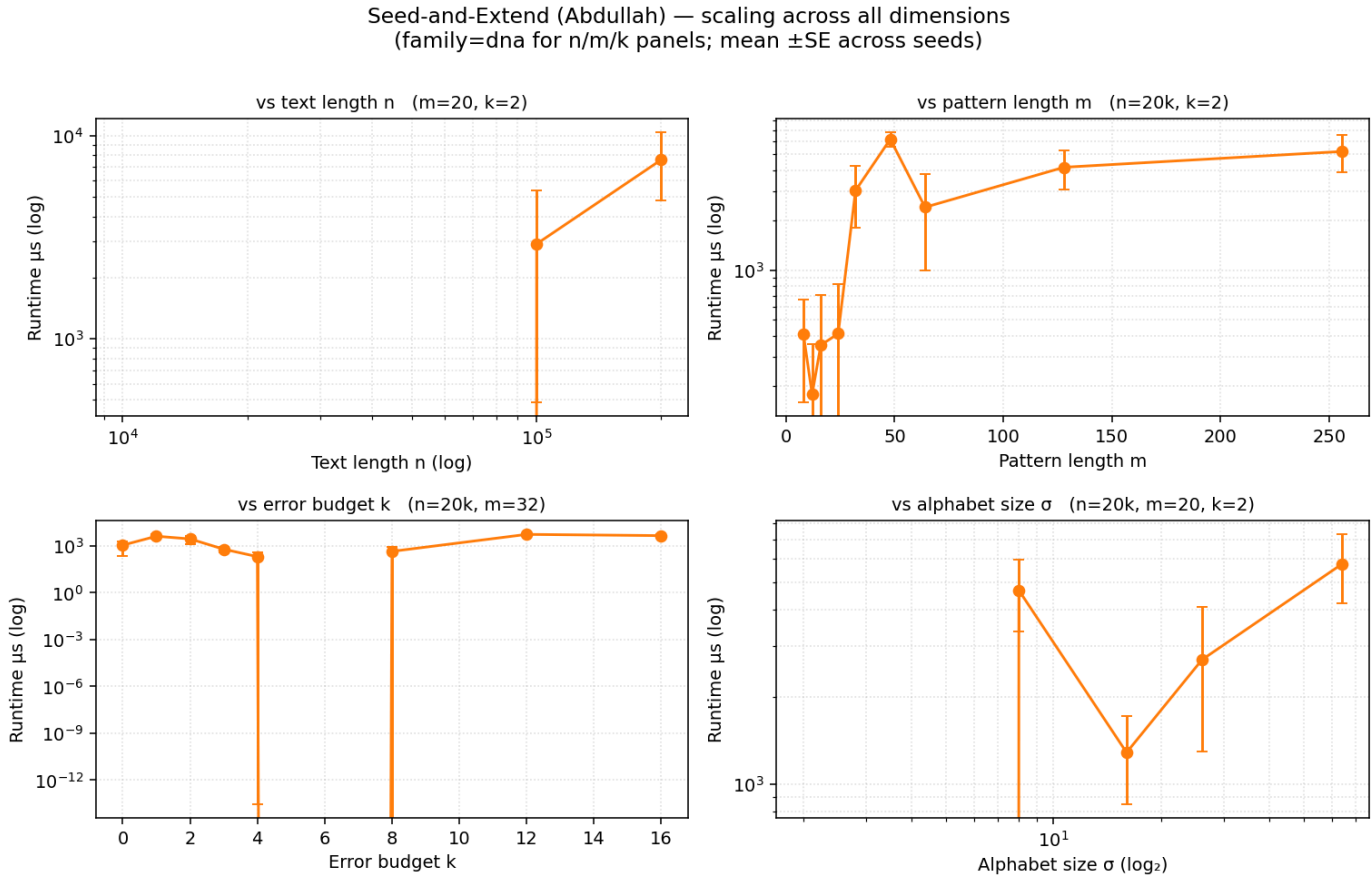
2. Seed lookup		
P 's q -grams: AC, CG, GT		
q -gram	hit	t_{start}
AC ($p=0$)	0, 4	0, 4
CG ($p=1$)	1	0
GT ($p=2$)	2	0
Unique: {0, 4}		

3. Verify		
t	$T[t..t+3]$	d_H
0	ACGT	0 ✓
4	ACAT	1 ✓
Matches {0, 4}		

Why it's correct (Hamming). if P matches T at position i with $\leq k$ mismatches, at least one q -gram of P must exactly match T near i (pigeonhole: $q \leq \lfloor m/(k + 1) \rfloor$).

My Results: Speed & the Trade-off

Speed — fastest by median of all 4 algorithms



Metric	Value
Median runtime	1.0 ms
Mean runtime	7.8 ms
P95 runtime	22ms
Empirical exponent on n	$\hat{\alpha} \approx 1.38$
Cases skipped	0 /

Trade-off — what does speed cost us?

Metric	Value
$\bar{P}^-$ (precision)	1.00
$\bar{R}$ (recall)	0.24
R at $k=1$	0.40
R at $k \geq 2$	~ 0

Why the recall collapse? The verifier computes *Hamming* distance, not *edit* distance — a pattern shifted by one indel has $d_H \sim m$ from the text, looking like noise.

Use when: substitutions only (DNA SNP), huge inputs, or as a candidate generator feeding an exact verifier.

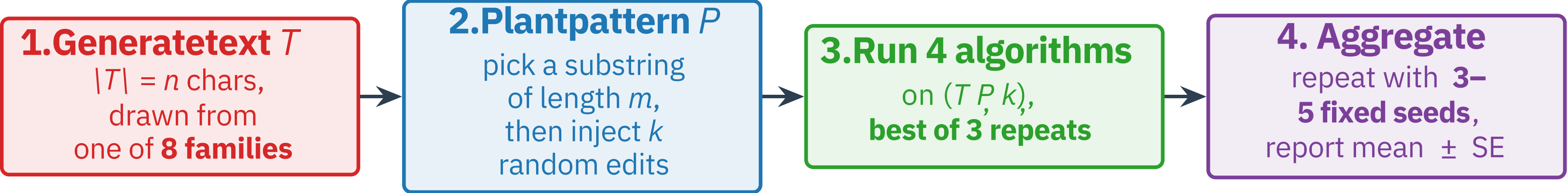
Avoid when: indels are likely (real biology, OCR), or any missed match is unacceptable.

Comparative Analysis

Four algorithms, one benchmark

Benchmark Setup: How Each Test Case is Generated

Every measurement in the next slides comes from this exact pipeline.



8 text families *input structure, not just size, is varied*

Synthetic (random):

dna	uniform $\{A, C, G, T\}$
english	uniform $a-z$
binary	uniform $\{0, 1\}$
repetitive	all A's, sparse T's
low_entropy_dna	biased 40/10/10/40 DNA
adversarial	+ motif repeats

Semi-real:

real_dna	<i>E. coli</i> Markov ($\approx 51\%$ GC)
real_english	Shakespeare <i>Hamlet</i> tiled

5 parameter sweeps *one variable at a time*

Vary	Range (others fixed)
family	all 8 ($n=5k, m=16, k=2$)
n	$10^4 \rightarrow 2 \times 10^5$ ($m=20, k=2$)
m	$8 \rightarrow 256$ ($n=20k, k=2$)
k	$0 \rightarrow 16$ ($n=20k, m=32$)
σ	$2 \rightarrow 64$ ($n=20k, m=20, k=2$)

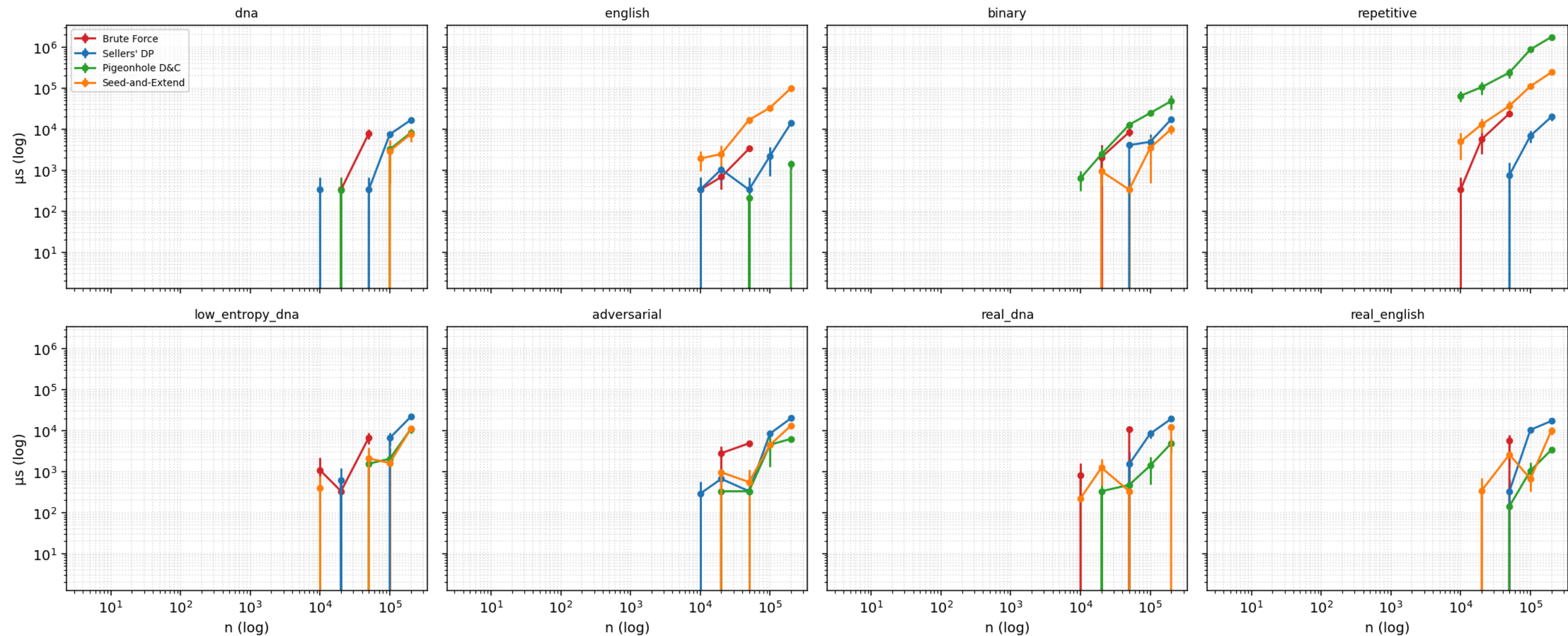
Seeds = {42, 7, 1729, 31337, 271828}

Runtime Scaling with n

Across all 8 input families.

$m=20, k=2$. Mean over 3 seeds, log-log axes.

Scaling with n by family ($m=20, k=2$; mean $\pm$ SE)



Empirical vs Theoretical Scaling on n

Fitted slope of $\log(\text{runtime})$ vs $\log(n)$, compared to each algorithm's theoretical exponent. .

Algorithm	Empirical	Theoretical	Δ	Verdict
Brute Force	3.45	1	+2.45	dominated by 3^k
Sellers' DP	1.37	1	+0.37	matches $O(nm)$
Pigeonhole D&C	1.41	1	+0.41	matches $O(n/k)$
Seed-and-Extend	1.38	1	+0.38	matches $O(n)$

Three of four algorithms sit between 1.3 and 1.5 — close to the theoretical linear regime, with cache and timer-floor effects pushing the slope above 1.

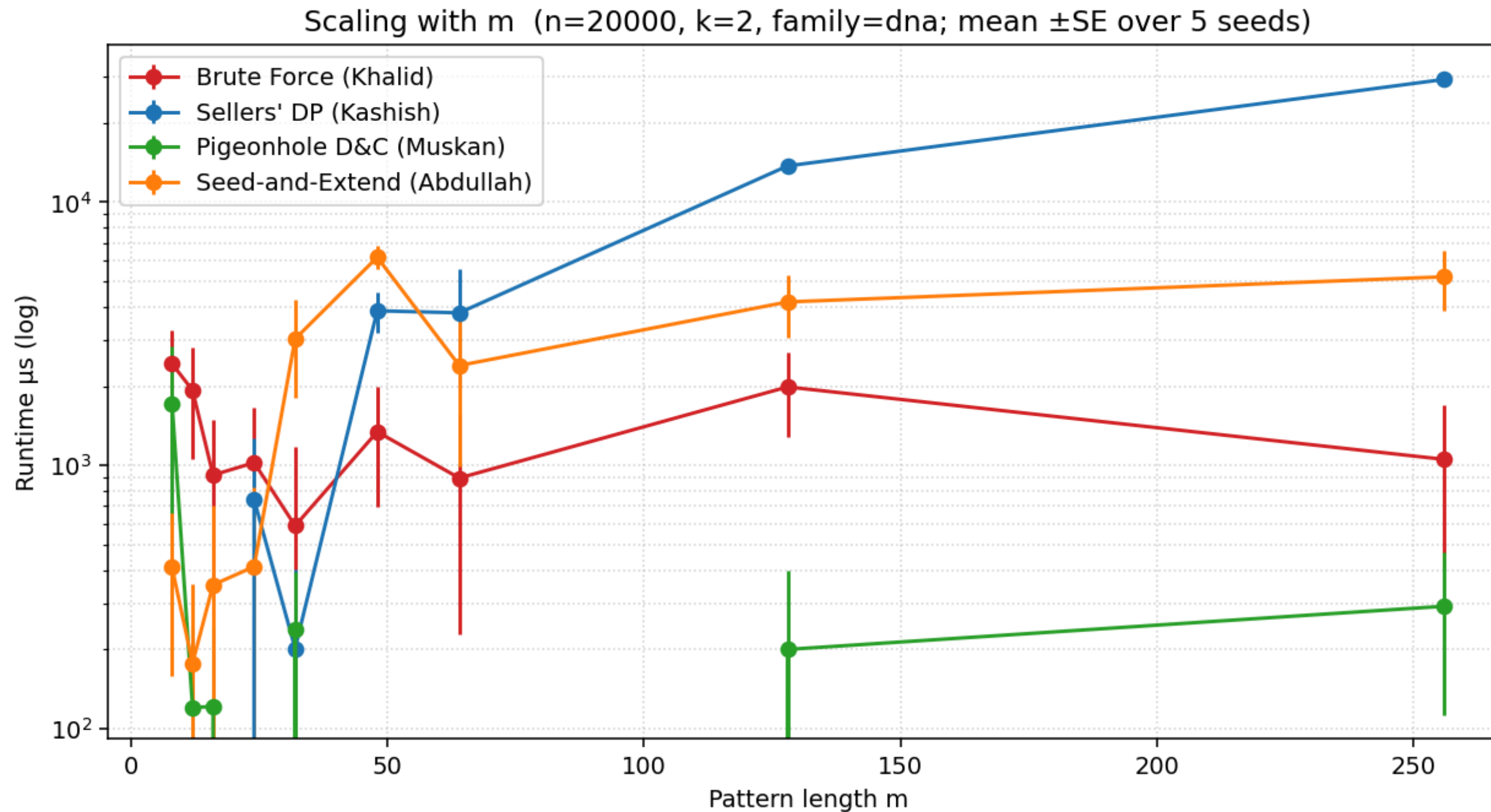
Brute Force at $\alpha^{\wedge} \approx 3.45$ confirms that the $3k$ factor dominates any apparent n -scaling.

Runtime Scaling with m

DNA, $n=20\ 000$, $k=2$.

Log-y axis.

Mean $\pm$ SE over 5 seeds..

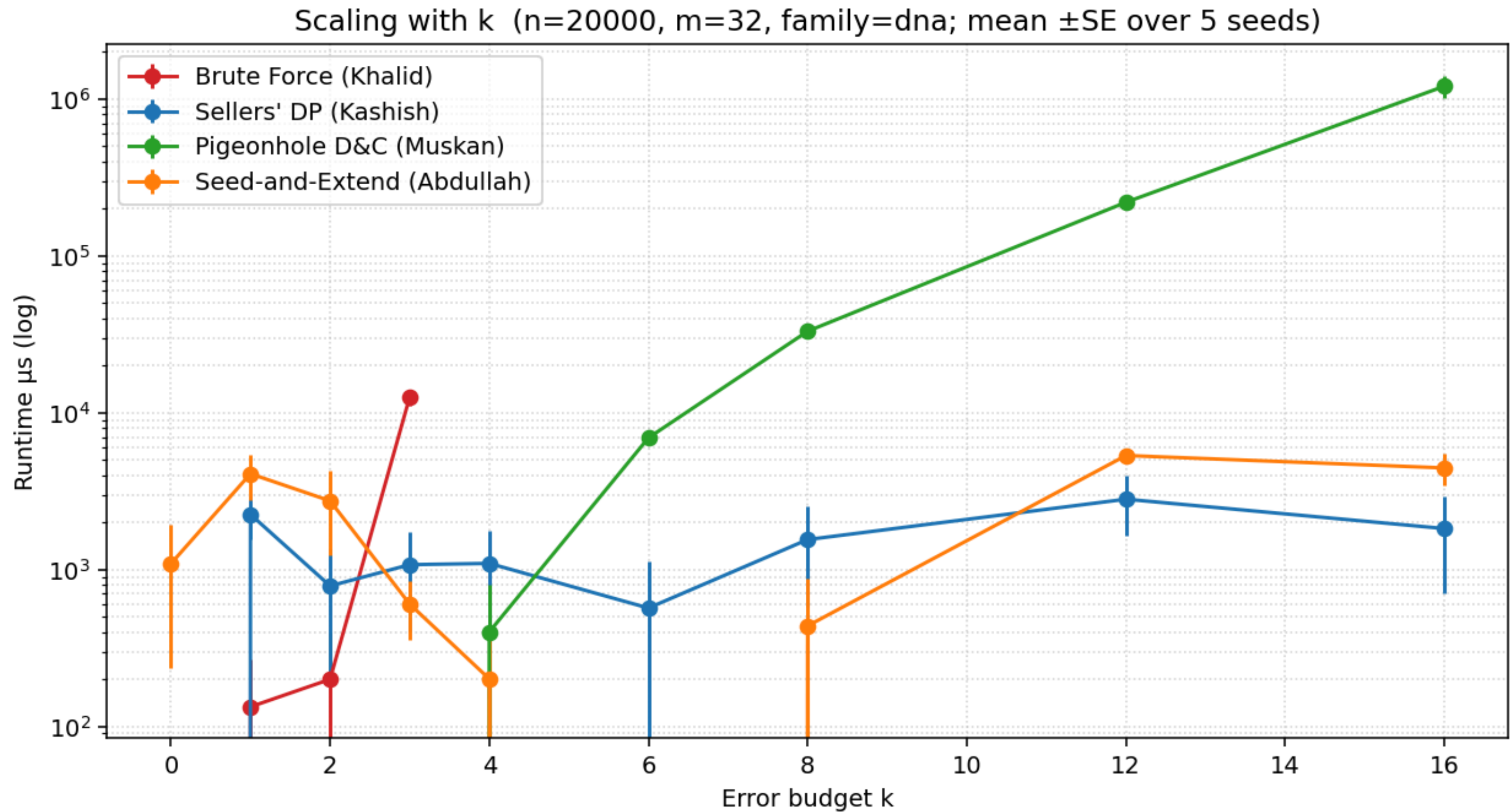


Sellers' DP is the only algorithm whose runtime is *linear in m* — exactly as $O(nm)$ predicts.

Seed-and-Extend stays almost flat: bigger m means longer seeds, which means *fewer* false candidates to verify.

Runtime Scaling with k – the Stress Test

DNA, $n=20\ 000$, $m=32$. Log-y axis. Mean $\pm$ SE over 5 seeds



Runtime vs k — The Numbers

Mean runtime (μ s) at each error budget k . DNA, $n=20\,000$, $m=32$.

k	0	2	4	8	12	16
Sellers' DP	0	786	1,095	1,553	2,810	1,826
Pigeonhole D&C	0	0	400	33,104	220,023	1,209,155
Seed-and-Extend	1,090	2,743	201	437	5,338	4,446
Brute Force	0	200	<i>skip</i>	<i>skip</i>	<i>skip</i>	<i>skip</i>

Pigeonhole degrades from 0 μ s at $k \leq 3$ to 1.21 seconds at $k = 16$ — a $>10^6\times$ slowdown across the sweep.

Sellers stays in the 1–3 ms band for every k — the price of $O(nm)$ is also its guarantee.

Brute Force skipped past $k = 3$: the $3k$ recursion times out.

Accuracy: Precision & Recall vs k

Quality of the matches each algorithm reports, relative to Sellers’ exact ground truth.

Table 5 — Precision (P) and recall (R) vs error budget k

Sellers' DP is the ground truth (windowed $\pm k$ 1-to-1 matching). DNA, $n=20k$, $m=32$.

k	Brute P	Sellers P	Pigeon P	Seed P	Brute R	Sellers R	Pigeon R	Seed R
0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
1	0.867	1.0	1.0	1.0	1.0	1.0	1.0	0.4
2	0.867	1.0	1.0	1.0	1.0	1.0	1.0	0.0
3	0.867	1.0	1.0	1.0	1.0	1.0	1.0	0.0
4	—	1.0	1.0	1.0	—	1.0	1.0	0.0
6	—	1.0	1.0	1.0	—	1.0	1.0	0.0
8	—	1.0	1.0	1.0	—	1.0	1.0	0.0
12	—	1.0	1.0	1.0	—	1.0	1.0	0.005
16	—	1.0	1.0	1.0	—	1.0	1.0	0.014

Accuracy Summary—Who Reports What

Mean precision $\bar{P}$ and recall $\bar{R}$ aggregated across all k .

Algorithm	$\bar{P}$	$\bar{R}$	Behaviour
Sellers' DP	1.00	1.00	exact for every k (ground truth)
Brute Force	0.90	0.94	exact when it runs (74 cases skipped)
Seed-and-Extend	1.00	0.24	no false positives, but Hamming-only
Pigeonhole D&C	1.00	1.00	exact for every k (matches Sellers)

Seed-and-Extend keeps $\bar{P} = 1.0$ (every reported match is real) but its recall collapses from 0.4 at $k = 1$ to 0.0 from $k = 2$ until k approaches m — it cannot see insertions or deletions..

Pigeonhole achieves $\bar{P} = \bar{R} = 1.0$ at every k — the divide-and-conquer filter is provably exact, and our DP verifier matches Sellers' ground truth match-for-match

The Verdict: Which Algorithm, When?

Each algorithm wins a different regime—pick the one that matches your inputs.

Sellers' DP $\bar{P}=\bar{R}=1.00$ | mean 3.3 ms

Use when: you need an exact answer and $n \lesssim 10^5$.

Predictable $O(nm)$. The reference algorithm.

Pigeonhole D&C $0 \mu\text{s}$ at $k \leq 4 \rightarrow 1.55 \text{ s}$ at $k=16$

Use when: error budget $k/m \lesssim 0.1$ (very small).

Fastest exact algorithm in its regime; fragile outside it.

Brute Force $\bar{P}=0.90$, $\bar{R}=0.94$ | 74 cases skipped

Use when: never in production — empirical exponent **3.45** rules it out.

Pedagogical baseline only.

Seed-and-Extend $\bar{P}=1.00$, $\bar{R}=0.24$ | median 1.0 ms

Use when: inputs are *huge* and substitution-only matches suffice.

The BLAST trade-off: throughput over recall.

The right algorithm depends on n , k , and how much recall you can afford to lose.